Practical - 15

Selenium WebDriver: Login Validation and Logout Testing

1. Steps

1. Open VS Code: Open your project folder and create a Python file named tutorial15.py.
2. Install Selenium: Open the VS Code Terminal and run: pip install selenium
3. Check Chrome: Make sure Google Chrome is installed. Selenium Manager normally manages the required driver automatically in recent Selenium versions.
4. Enter the code: Copy the complete program shown in Section 2 into tutorial15.py and save the file.
5. Run the program: Click Run in VS Code, or run: python tutorial15.py.

2. Python Program

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.common.exceptions import (
    NoSuchElementException,
    TimeoutException,
    ElementNotInteractableException,
    StaleElementReferenceException,
    WebDriverException
)

# Website details
URL = "https://www.saucedemo.com/"
VALID_USERNAME = "standard_user"
VALID_PASSWORD = "secret_sauce"

# Start browser
def start_browser():
    driver = webdriver.Chrome()
    driver.maximize_window()
    return driver

# Login function
def login(driver, username, password):
    driver.get(URL)
    wait = WebDriverWait(driver, 10)

    username_box = wait.until(
        EC.visibility_of_element_located((By.ID, "user-name"))
    )
    password_box = wait.until(
        EC.visibility_of_element_located((By.ID, "password"))
    )

    username_box.clear()
    username_box.send_keys(username)
    password_box.clear()
    password_box.send_keys(password)

    wait.until(
        EC.element_to_be_clickable((By.ID, "login-button"))
    ).click()

# Check whether login reached the inventory page
def is_logged_in(driver):
    try:
        WebDriverWait(driver, 5).until(
            EC.url_contains("inventory.html")
        )
        return True
    except TimeoutException:
        return False

# Read the login error message
def get_login_error(driver):
    try:
        return WebDriverWait(driver, 5).until(
            EC.visibility_of_element_located(
```

```
                (By.CSS_SELECTOR, "[data-test='error']")
            )
        ).text
    except (TimeoutException, NoSuchElementException):
        return ""

# TC-01: Valid login
def test_valid_login():
    driver = start_browser()
    try:
        login(driver, VALID_USERNAME, VALID_PASSWORD)
        assert is_logged_in(driver), "Valid login did not open inventory page"
        print("TC-01 Valid Login: PASS")
    finally:
        driver.quit()

# TC-02: Invalid password
def test_invalid_password():
    driver = start_browser()
    try:
        login(driver, VALID_USERNAME, "wrong_password")
        assert not is_logged_in(driver), "Invalid password was accepted"
        assert get_login_error(driver), "Expected login error was not displayed"
        print("TC-02 Invalid Password: PASS")
    finally:
        driver.quit()

# TC-03: Invalid username
def test_invalid_username():
    driver = start_browser()
    try:
        login(driver, "wrong_user", VALID_PASSWORD)
        assert not is_logged_in(driver), "Invalid username was accepted"
        assert get_login_error(driver), "Expected login error was not displayed"
        print("TC-03 Invalid Username: PASS")
    finally:
        driver.quit()

# TC-04: Empty credentials
def test_empty_credentials():
    driver = start_browser()
    try:
        login(driver, "", "")
        assert not is_logged_in(driver), "Empty credentials were accepted"
        assert get_login_error(driver), "Expected validation message was not displayed"
        print("TC-04 Empty Credentials: PASS")
    finally:
        driver.quit()

# TC-05: Logout
def test_logout():
    driver = start_browser()
    try:
        login(driver, VALID_USERNAME, VALID_PASSWORD)
        assert is_logged_in(driver), "Login failed before logout test"

        wait = WebDriverWait(driver, 10)
        wait.until(
            EC.element_to_be_clickable((By.ID, "react-burger-menu-btn"))
        ).click()
        wait.until(
            EC.element_to_be_clickable((By.ID, "logout_sidebar_link"))
        ).click()

        wait.until(EC.visibility_of_element_located((By.ID, "login-button")))
        assert URL in driver.current_url, "Logout did not return to login page"
        print("TC-05 Logout: PASS")
    finally:
        driver.quit()

if __name__ == "__main__":
    tests = [
        test_valid_login,
        test_invalid_password,
        test_invalid_username,
        test_empty_credentials,
        test_logout
    ]

    for test in tests:
        try:
            test()
```

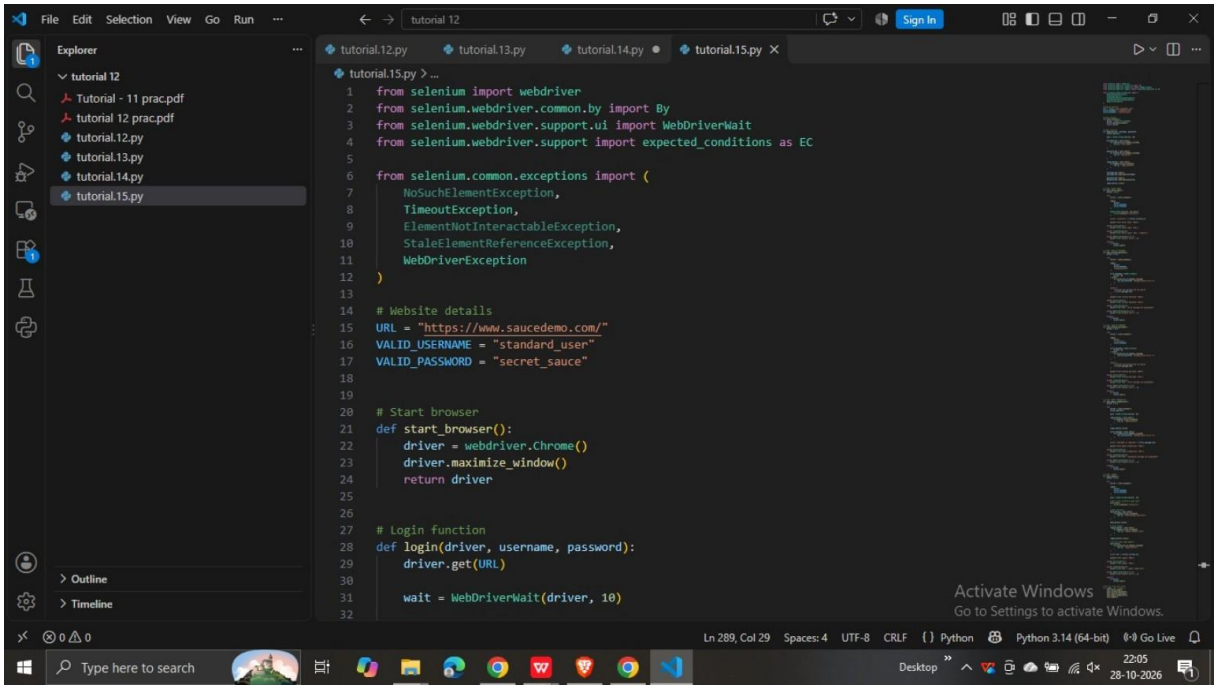
```
        except AssertionError as error:
            print(f"{test.__name__}: FAIL - {error}")
        except (WebDriverException, TimeoutException,
                NoSuchElementException, ElementNotInteractableException,
                StaleElementReferenceException) as error:
            print(f"{test.__name__}: FAIL - {type(error).__name__}: {error}")
```

3. Explanation of the Code

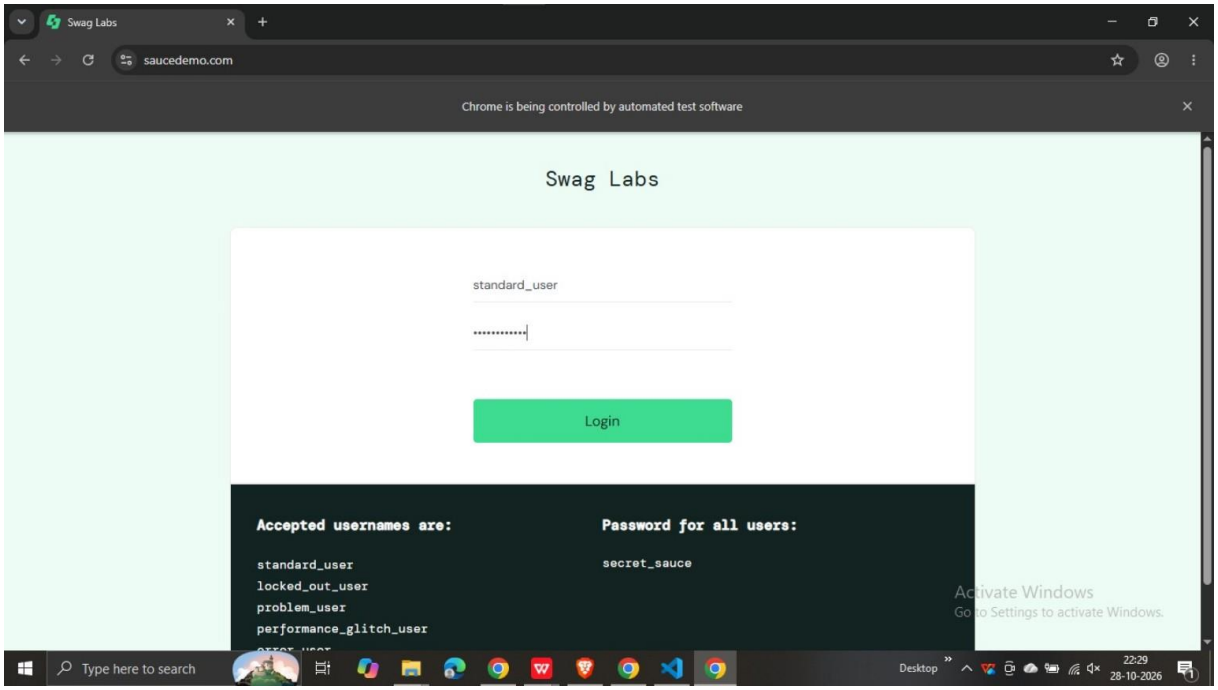
- webdriver.Chrome() — Starts a Chrome browser controlled by Selenium.
- driver.maximize_window() — Maximizes the browser window.
- driver.get(URL) — Opens the SauceDemo login page.
- WebDriverWait and expected_conditions — Wait for page elements to become visible or clickable before interacting with them.
- login(driver, username, password) — Enters the supplied username and password, then clicks Login.
- is_logged_in(driver) — Checks whether the browser reached inventory.html after login.
- get_login_error(driver) — Reads the validation or login error displayed by the website.
- TC-01 Valid Login — Confirms that the valid credentials open the inventory page.
- TC-02 Invalid Password — Confirms that a wrong password does not log in and an error is displayed.
- TC-03 Invalid Username — Confirms that a wrong username does not log in and an error is displayed.
- TC-04 Empty Credentials — Confirms that blank username and password are rejected.
- TC-05 Logout — Logs in, opens the menu, selects Logout, and verifies that the login page appears again.
- driver.quit() — Closes the browser and ends the WebDriver session after each test.

4. Screenshots

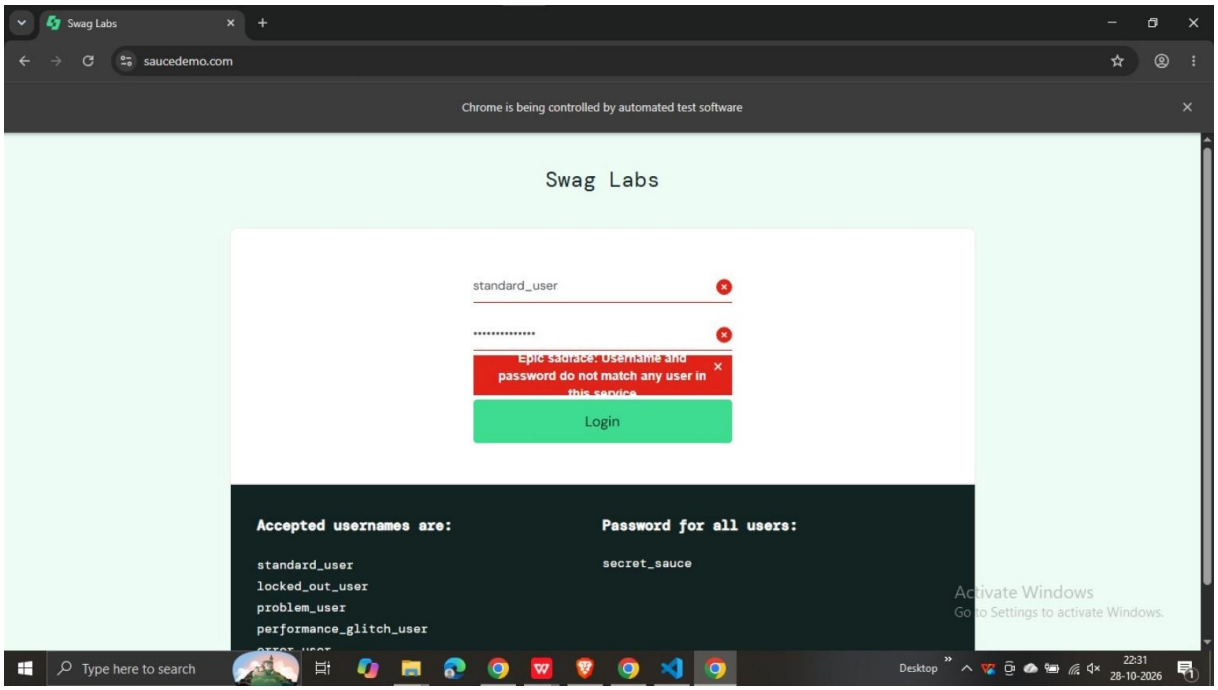
Screenshot 1: Python code in VS Code.



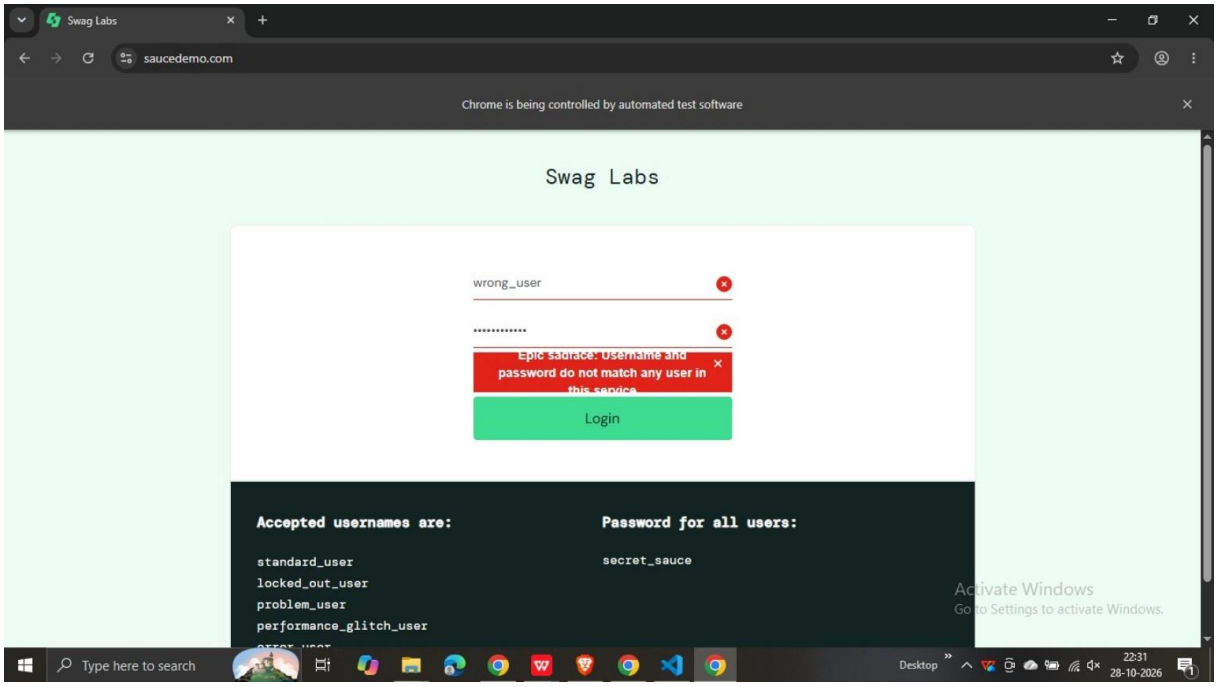
Screenshot 2: SauceDemo login page with the test credentials.



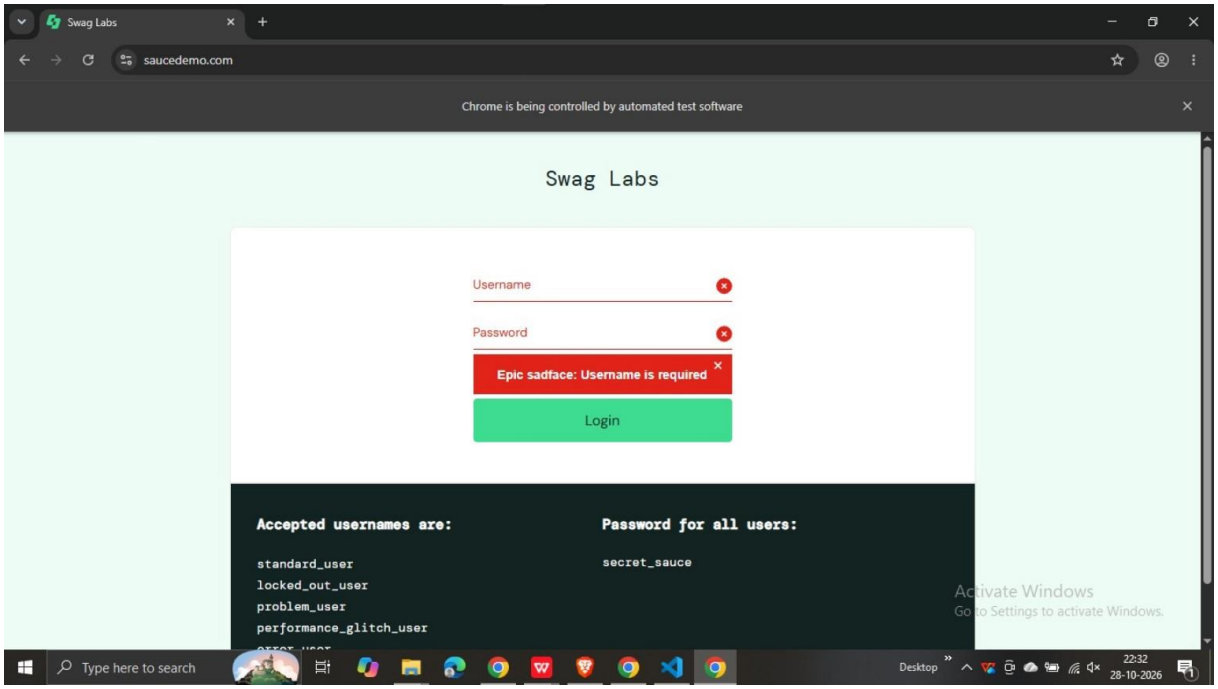
Screenshot 3: Login validation message for incorrect credentials.



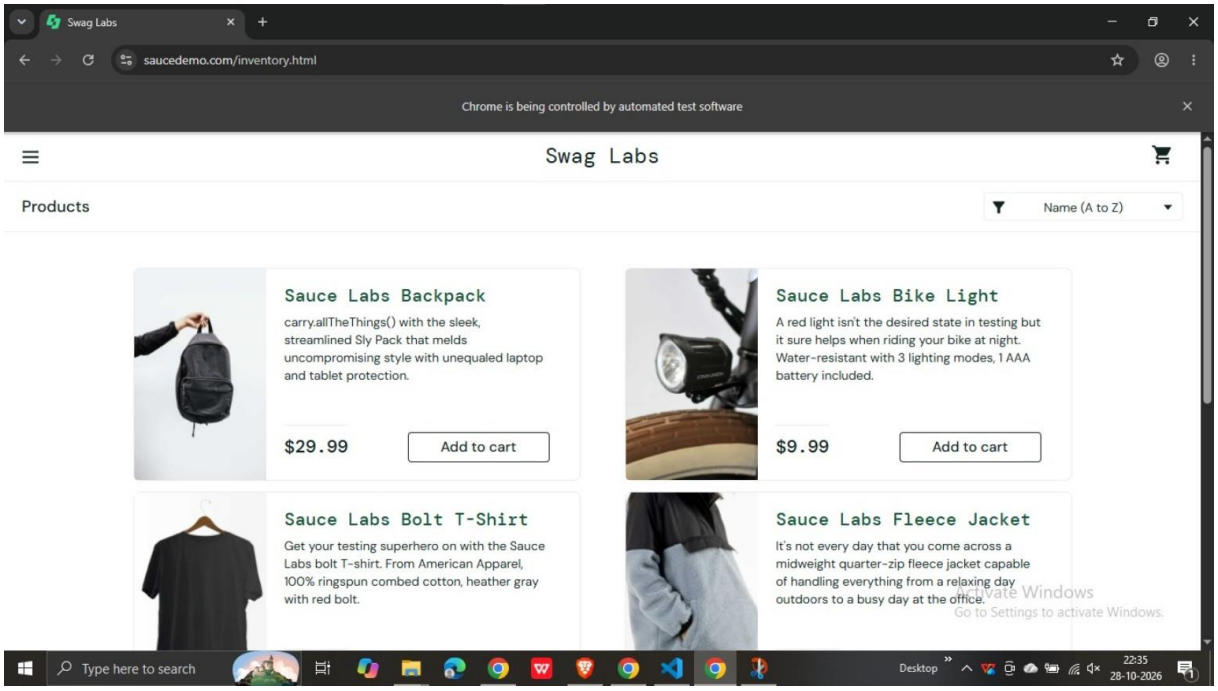
Screenshot 4: Invalid username and password validation.



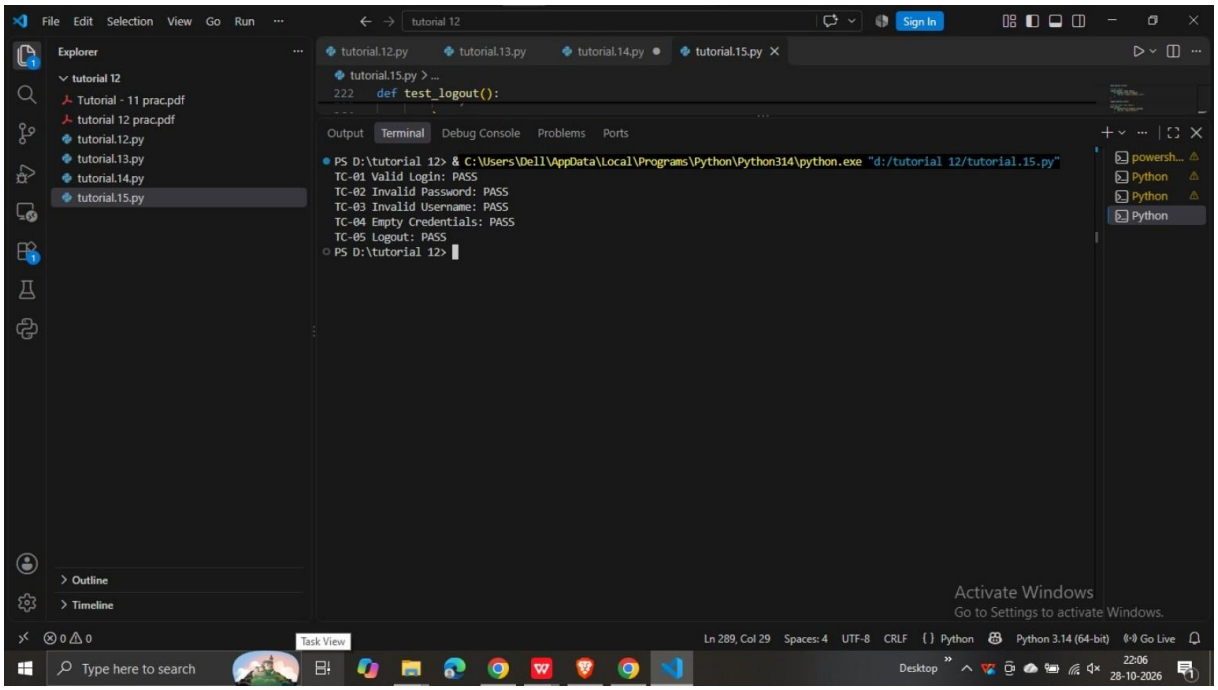
Screenshot 5: Empty-credentials validation message.



Screenshot 6: SauceDemo inventory page after successful login.



Screenshot 7: VS Code terminal showing the five test results.



5. Expected Output

```
TC-01 Valid Login: PASS
TC-02 Invalid Password: PASS
TC-03 Invalid Username: PASS
TC-04 Empty Credentials: PASS
TC-05 Logout: PASS
```

6. Result / Conclusion

The Selenium WebDriver program tests the SauceDemo login page using valid, invalid, and empty credentials. It also verifies the logout operation. Each test prints PASS when its expected behavior is observed; otherwise, the test prints a failure message.